

UNIVERSIDAD NACIONAL DEL ALTIPLANO

**FACULTAD DE INGENIERIA MECANICA ELECTRICA,
ELECTRONICA Y DE SISTEMAS.**

ESCUELA PROFESIONAL DE INGENIERIA DE SISTEMAS



INFORME DE COMPARACION DE ALGORITMOS DE ORDENAMIENTO

ESTUDIANTE:

APAZA RAMIREZ JIMMY EDSON

CURSO:

PROGRAMACION COMPETITIVA

CICLO: "VI" GRUPO: "A"

DOCENTE:

ING. ACEITUNO ROJO MIGUEL ROMILIO

PUNO - PERU

2026

1. INTRODUCCION

El ordenamiento de datos es una de las operaciones más fundamentales en ciencias de la computación. Prácticamente cualquier sistema que gestione colecciones de información necesita, en algún momento, organizarla de manera eficiente. Elegir el algoritmo correcto puede representar la diferencia entre un sistema que responde en milisegundos y uno que tarda varios segundos.

SortBench es una herramienta de benchmarking desarrollada en Python con una interfaz web moderna que permite al usuario generar arreglos aleatorios de tamaño y rango configurables, y comparar el desempeño de diez algoritmos de ordenamiento clásicos de forma simultánea. Los resultados se presentan ordenados por tiempo de ejecución en nanosegundos, junto con la cantidad de pasos (comparaciones e intercambios) realizados por cada algoritmo.

1.1 Objetivos

- Implementar diez algoritmos de ordenamiento desde cero en Python, sin usar funciones de ordenamiento nativas.
- Medir con precisión de nanosegundos el tiempo de ejecución de cada algoritmo.
- Contabilizar la cantidad de pasos (operaciones elementales) realizados.
- Presentar los resultados en una interfaz web clara, ordenada y visualmente atractiva.
- Facilitar la comprensión práctica de las diferencias de rendimiento entre algoritmos.

1.2 Alcance

El sistema compara diez algoritmos sobre el mismo arreglo generado aleatoriamente, permitiendo al usuario definir la cantidad de elementos y el rango numérico. Soporta enteros negativos y positivos, y cantidades de hasta 100,000 elementos.

2. Tecnologías y Librerías Utilizadas

El proyecto está construido con tecnologías minimalistas, priorizando claridad y portabilidad. No requiere instalación de frameworks pesados ni bases de datos.

2.1 Python 3

Rol: Lenguaje principal del backend y motor de los algoritmos.

Python fue elegido por su claridad sintáctica, lo que permite implementar los algoritmos de manera legible y educativa. Su módulo estándar

`time.perf_counter_ns()` provee medición de tiempo con precisión de nanosegundos, ideal para comparaciones de rendimiento a baja escala.

Versión requerida: Python 3.8 o superior.

2.2 Flask

Rol: Servidor web ligero que actúa como puente entre la interfaz HTML y el código Python.

Flask es un microframework web para Python. En este proyecto cumple dos funciones: servir el archivo

`index.html` al navegador, y exponer el endpoint `POST /benchmark` que recibe los parámetros del usuario, ejecuta los algoritmos y devuelve los resultados en formato JSON.

```
pip install flask
```

Flask no agrega ninguna lógica de ordenamiento; simplemente actúa como intermediario HTTP.

2.3 HTML5 + CSS3 + JavaScript (Vanilla)

Rol: Interfaz de usuario completa, contenida en un único archivo `index.html`.

Se optó por no usar frameworks de frontend (React, Vue, etc.) para mantener el proyecto simple y sin dependencias adicionales. La interfaz usa:

- **CSS Variables:** para gestionar el tema visual de manera centralizada.

- **Fetch API:** para comunicarse con el backend Flask de forma asíncrona (sin recargar la página).
- **Animaciones CSS:** para revelar las filas de resultados progresivamente.
- **Google Fonts (JetBrains Mono + Syne):** tipografías cargadas desde CDN para la estética de terminal.

2.4 Librerías estándar de Python usadas

Módulo	Uso en el proyecto
random	Generar el arreglo aleatorio con randint(lo, hi)
time	Medir tiempo con perf_counter_ns() en nanosegundos
webbrowser	Abrir el navegador automáticamente al iniciar
threading	Abrir el navegador en hilo separado sin bloquear Flask
os	Gestión de rutas de archivos estáticos

3. Algoritmos de Ordenamiento

Se implementaron diez algoritmos clásicos. Cada uno recibe una copia independiente del arreglo original (para no afectar las mediciones de los demás), y retorna la lista ordenada junto con la cantidad de pasos contabilizados.

3.1 Bubble Sort (Ordenamiento Burbuja)

Compara elementos adyacentes e intercambia aquellos que están en el orden incorrecto, repitiendo el proceso hasta que el arreglo esté ordenado. En cada pasada, el elemento mayor "burbujea" hacia el final.

Complejidad	Mejor caso	Caso promedio	Peor caso	Espacio
Tiempo	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$

3.2 Selection Sort (Ordenamiento por Selección)

Divide el arreglo en una parte ordenada y otra no ordenada. En cada iteración, selecciona el elemento mínimo de la parte no ordenada y lo intercambia con el primer elemento de esa parte.

Complejidad	Mejor caso	Caso promedio	Peor caso	Espacio
Tiempo	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$

3.3 Insertion Sort (Ordenamiento por Inserción)

Construye el arreglo ordenado de izquierda a derecha, tomando cada elemento y ubicándolo en su posición correcta dentro de la parte ya ordenada, desplazando los demás hacia la derecha.

Complejidad	Mejor caso	Caso promedio	Peor caso	Espacio
Tiempo	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$

Nota: Es el más eficiente de los algoritmos $O(n^2)$ en arreglos pequeños o casi ordenados.

3.4 Shell Sort

Variante mejorada de Insertion Sort. Compara elementos separados por un "gap" que va reduciéndose progresivamente, permitiendo mover elementos grandes a su posición en menos pasos que el Insertion Sort clásico.

Complejidad	Mejor caso	Caso promedio	Peor caso	Espacio
Tiempo	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(1)$

3.5 Merge Sort (Ordenamiento por Mezcla)

Aplica la estrategia divide y vencerás: divide el arreglo en mitades recursivamente hasta llegar a elementos individuales, y luego las fusiona (merge) en orden. Es estable y garantiza $O(n \log n)$ en todos los casos.

Complejidad	Mejor caso	Caso promedio	Peor caso	Espacio
Tiempo	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

3.6 Quick Sort (Ordenamiento Rápido)

Elige un elemento "pivote" y particiona el arreglo en dos subconjuntos: elementos menores al pivote y elementos mayores. Aplica recursión en cada subconjunto. En promedio es el más rápido en la práctica, aunque su peor caso es $O(n^2)$ si el pivote es siempre el mínimo o máximo.

Complejidad	Mejor caso	Caso promedio	Peor caso	Espacio
Tiempo	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$

3.7 Heap Sort (Ordenamiento por Montículo)

Construye un montículo máximo (max-heap) con todos los elementos, extrae repetidamente el máximo y lo coloca al final del arreglo. Garantiza $O(n \log n)$ y no requiere memoria adicional.

Complejidad	Mejor caso	Caso promedio	Peor caso	Espacio
Tiempo	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

3.8 Tim Sort

Algoritmo híbrido que combina Insertion Sort y Merge Sort. Divide el arreglo en bloques pequeños (runs de 32 elementos), los ordena con Insertion Sort y luego los fusiona con Merge Sort. Es el algoritmo interno de Python y Java. Es extremadamente eficiente en datos del mundo real.

Complejidad	Mejor caso	Caso promedio	Peor caso	Espacio
Tiempo	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

3.9 Counting Sort (Ordenamiento por Conteo)

No realiza comparaciones entre elementos. Cuenta la frecuencia de cada valor dentro del rango y reconstruye el arreglo ordenado. Es extremadamente rápido cuando el rango de valores (k) es pequeño relativo a n .

Complejidad	Mejor caso	Caso promedio	Peor caso	Espacio
Tiempo	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$

Limitación: Solo funciona con enteros. Su eficiencia depende del rango $k = \text{máximo} - \text{mínimo}$. Soporta negativos mediante un desplazamiento (offset).

3.10 Radix Sort (Ordenamiento por Dígitos)

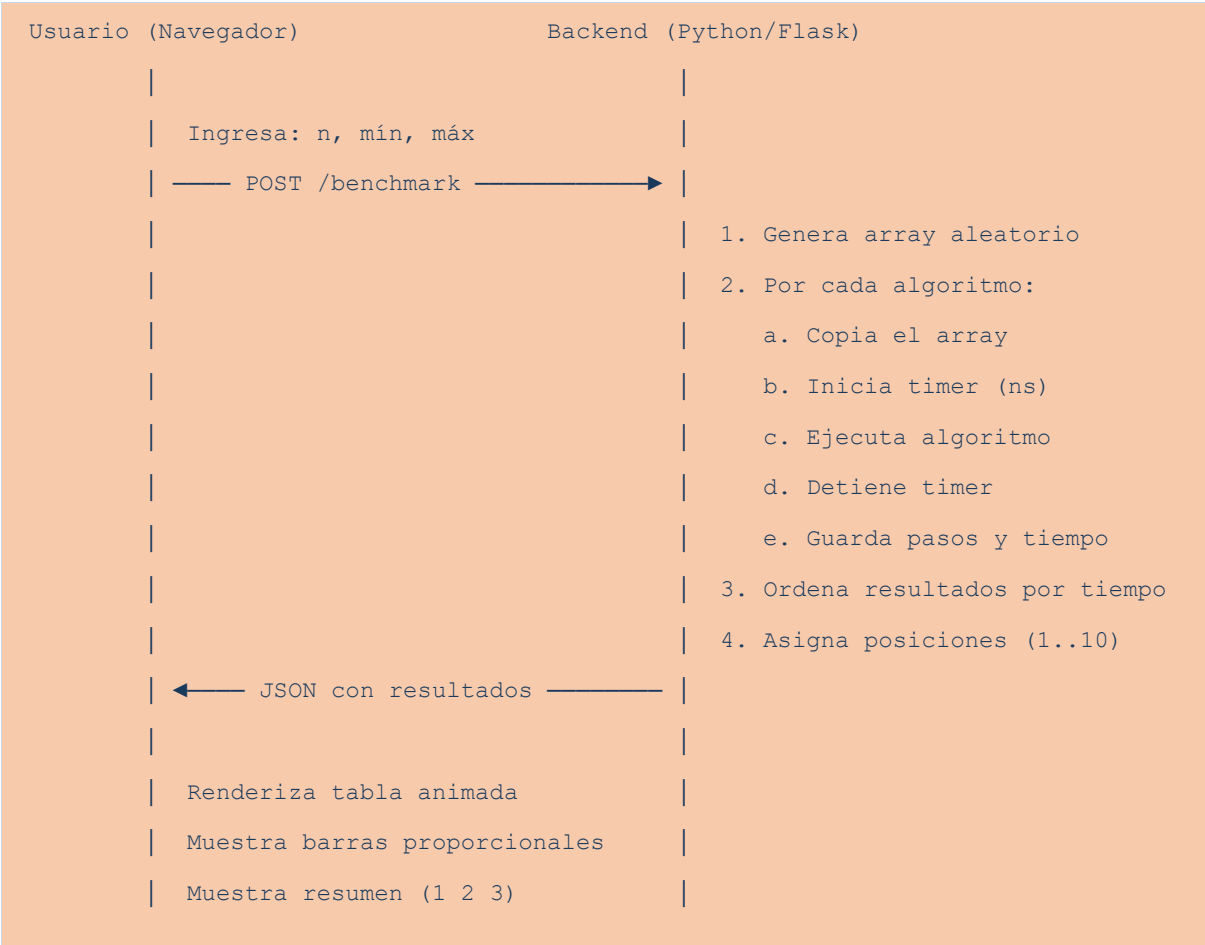
Ordena los números dígito por dígito, de menos significativo a más significativo, usando Counting Sort como subrutina en cada dígito. Es lineal cuando el número de dígitos k es constante.

Complejidad	Mejor caso	Caso promedio	Peor caso	Espacio
Tiempo	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$

4. Arquitectura del Sistema

SortBench sigue una arquitectura cliente-servidor minimalista de dos capas, donde toda la lógica de ordenamiento reside en el backend Python y la presentación en el frontend HTML.

4.1 Diagrama de Flujo



4.2 Endpoint de la API

Campo	Detalle
Ruta	POST /benchmark
Entrada	JSON: { "n": 100, "lo": 1, "hi": 999 }
Salida	JSON: { "array": [...], "results": [...] }

Campo	Detalle
Tiempo med.	perf_counter_ns() — precisión de nanosegundos

4.3 Estructura de Respuesta JSON

```
{
  "array": [42, 17, 83, ...],      // arreglo generado
  "results": [
    {
      "position": 1,              // posición (1 = más rápido)
      "name":      "Quick Sort",  // nombre del algoritmo
      "steps":     1248,          // pasos realizados
      "time_ns":   45210         // tiempo en nanosegundos
    },
    ...
  ]
}
```

5. Análisis de Resultados

El comportamiento de los algoritmos varía significativamente según el tamaño del arreglo y el rango de valores. A continuación se describen los patrones observados.

5.1 Arreglos pequeños ($n < 50$)

En arreglos pequeños, la diferencia entre algoritmos es mínima y los resultados pueden variar entre ejecuciones por el overhead del sistema operativo. Los algoritmos simples como Insertion Sort y Shell Sort suelen competir favorablemente con los más complejos.

Observación: Counting Sort y Radix Sort pueden ser más lentos en arreglos pequeños debido a su overhead de inicialización de arreglos auxiliares.

5.2 Arreglos medianos ($50 \leq n \leq 5,000$)

La diferencia entre algoritmos $O(n^2)$ y $O(n \log n)$ empieza a ser notable. Quick Sort, Merge Sort, Heap Sort y Tim Sort mantienen tiempos bajos, mientras que Bubble Sort, Selection Sort e Insertion Sort empiezan a crecer cuadráticamente.

5.3 Arreglos grandes ($n > 5,000$)

Los algoritmos $O(n^2)$ se vuelven impracticamente lentos. Bubble Sort con 10,000 elementos puede tardar cientos de veces más que Quick Sort. Counting Sort y Radix Sort brillan cuando el rango de valores (k) es razonablemente pequeño.

Algoritmo	$n=100$	$n=1,000$	$n=10,000$	Escala
Bubble Sort	muy rápido	lento	muy lento	$O(n^2)$
Selection Sort	rápido	lento	muy lento	$O(n^2)$
Insertion Sort	rápido	lento	muy lento	$O(n^2)$
Shell Sort	rápido	rápido	moderado	$O(n \log n)$
Merge Sort	rápido	rápido	rápido	$O(n \log n)$
Quick Sort	rápido	muy rápido	muy rápido	$O(n \log n)$

Algoritmo	n=100	n=1,000	n=10,000	Escala
Heap Sort	rápido	rápido	rápido	$O(n \log n)$
Tim Sort	rápido	muy rápido	muy rápido	$O(n \log n)$
Counting Sort	variable	muy rápido	muy rápido	$O(n+k)$
Radix Sort	variable	muy rápido	muy rápido	$O(nk)$

6. Instrucciones de Ejecución

6.1 Requisitos previos

- Python 3.8 o superior instalado.
- Flask instalado: `pip install flask`
- Navegador web moderno (Chrome, Firefox, Edge).

6.2 Pasos para ejecutar

1. Descargar o clonar el repositorio del proyecto.
2. Abrir una terminal en la carpeta `COMPARACION/`.
3. Ejecutar: `python app.py` (o doble clic en `run.bat` en Windows).
4. El navegador se abrirá automáticamente en `http://localhost:5050`.
5. Ingresar los parámetros y presionar "Ejecutar Comparación".

6.3 Uso de la interfaz

Una vez abierta la interfaz web:

Campo	Descripción	Ejemplo
Cantidad de números	Tamaño del arreglo a generar	500
Valor mínimo	Límite inferior del rango	-100
Valor máximo	Límite superior del rango	999

Al ejecutar, la página muestra el arreglo generado (como chips desplazables), la tabla de resultados ordenada por tiempo con barras proporcionales, y un panel de resumen con el algoritmo más rápido, el más lento y el factor de diferencia entre ambos.

7. Conclusiones

El desarrollo de SortBench permitió comprobar de manera práctica y visual los conceptos teóricos de complejidad algorítmica estudiados en clase. Las conclusiones más relevantes son:

- **No existe un algoritmo universalmente óptimo.** La elección depende del tamaño del arreglo, el rango de valores y el estado inicial de los datos.
- **Los algoritmos $O(n^2)$ son viables solo para n pequeño.** Bubble Sort, Selection Sort e Insertion Sort se degradan rápidamente a partir de los 1,000 elementos.
- **Quick Sort y Tim Sort lideran en la práctica.** Para arreglos aleatorios de tamaño mediano a grande, estos dos algoritmos son consistentemente los más rápidos.
- **Counting Sort y Radix Sort son los más rápidos cuando el rango es acotado.** Al no realizar comparaciones, su crecimiento es lineal, pero requieren memoria adicional y solo funcionan con enteros.
- **La medición en nanosegundos revela diferencias imperceptibles en n pequeño.** Para arreglos de menos de 20 elementos, cualquier algoritmo responde en microsegundos y las diferencias son insignificantes.

Este proyecto demuestra que la teoría de algoritmos tiene impacto directo y medible en el rendimiento de sistemas reales, y que herramientas visuales como SortBench facilitan enormemente la comprensión de estos conceptos.